



# Stop Blaming the AI: How Disciplined Engineering Makes Generative AI Work

Kurt Antony Richardson 

## From the Editor

What is the key to scaling up the impressive capabilities of AI-assisted coding into production quality? To paraphrase a classic film, we've always had the key, we've had it all along, says our author Kurt Richardson: It is systems engineering discipline. And this also has important implications for how we should be managing our engineering workforce, he argues.— *John Favaro*

**A GROWING CHORUS** in the tech industry is sounding the alarm: Generative AI is creating a tsunami of technical debt. Developers are prompting their way to fragile, unmaintainable codebases. The phenomenon even has a name: “vibe coding,” coined by AI researcher Andrej Karpathy, who described it as “fully giv[ing] in to the vibes, embrace[ing] exponentials, and forget[ting] that the code even exists.”<sup>a</sup>

The critics aren't wrong about the problem. But they're catastrophically wrong about the diagnosis.

<sup>a</sup>A. Karpathy, “There's a new kind of coding I call 'vibe coding'...” X (formerly Twitter), Feb. 3, 2025. [Online]. Available: <https://x.com/karpathy/status/1886192184808149383>

To be fair, vibe coding has legitimate uses. Borg, Bjarnason, and Hedin recently demonstrated that vibe coding can be remarkably effective for rapid prototyping—enabling high-fidelity, executable prototypes that communicate product vision, validate requirements, and align teams far faster than traditional methods.<sup>1</sup> Their research identifies five distinct prototyping purposes—external communication, internal alignment, validation and testing, incremental development, and exploration—where the speed and accessibility of vibe coding deliver genuine value, even for nontechnical practitioners. The danger isn't vibe coding itself; it's mistaking a prototyping technique for a production development methodology.

Generative AI is remarkably good at writing code. The issue is that

many people are using it in ways that virtually guarantee failure—and the solution has been hiding in plain sight for decades. It's called systems engineering. When you pair disciplined engineering with generative AI, the results are transformative.

## The Limitation Nobody Wants to Talk About

Before we get to the solution, we need to understand the core technical constraint that makes vibe coding so dangerous for production systems: context dilution.

Large language models operate within a finite context window. As that window fills with thousands of lines of existing code, architectural decisions, and the sprawling state of a large application, the model's ability to reason coherently degrades.

It's a fundamental property of how attention mechanisms work in transformer architectures.

When you ask an LLM to “add user authentication to my app” and that app is already 50,000 lines of interconnected code, you're asking the model to comprehend a complex system while generating code that integrates cleanly with it. The model doesn't “understand” the codebase the way an experienced developer does. It's pattern-matching across a diluted sea of tokens, and output quality degrades proportionally with input complexity.

Anyone with a working understanding of these models would recognize this as a serious constraint for large-scale development. And yet the prevailing approach has been to ignore it—to treat LLMs as omniscient coding oracles. This is not a failure of artificial intelligence. It is a failure of engineering judgment.

### Working With the Grain

Here's what we've learned building production systems with generative AI at the core of our development workflow: AI-assisted code generation doesn't replace engineering discipline—it amplifies whatever discipline you bring. Bring none, and it amplifies chaos. Bring rigor, and it amplifies productivity in ways difficult to overstate.

Our guiding principle is simple: *Give the AI well-defined, manageable problems to solve.* That's the insight separating teams shipping reliable AI-generated code from teams drowning in technical debt.

In practice, this means doing something that software engineering has always demanded but that vibe coding explicitly rejects: planning before building.

## The Process

### Phase 1: Systems Engineering and Decomposition

Before generating code, we invest in upfront systems engineering—identifying core functionality, defining clean interfaces, and decomposing the system into discrete, independently implementable components. Generative AI excels at supporting this phase: requirements analysis, interface definition, architecture decomposition. These are tasks where LLMs add tremendous value because the context is focused and the output is structured reasoning.

The goal is a set of microservices, and we mean “micro.” Each service encapsulates a single coherent function behind a well-defined interface, typically no more than a couple thousand lines—small enough to be fully comprehended within a single, focused AI generation context. This isn't arbitrary; it's a direct response to context dilution.

### Phase 2: Requirements and Implementation Planning

With architecture defined, we write software requirements for each microservice and establish a pull request schedule. Each PR is scoped to a manageable increment. AI excels here too—given a well-defined interface and clear behavioral intent, an LLM can generate detailed requirements, identify test cases, and suggest implementation sequences with impressive quality.

### Phase 3: Incremental, Verified Code Generation

We have AI generate code for each PR—and we test between each step. This is nonnegotiable. Every increment is verified against requirements before the next begins. This catches

errors early and keeps the AI's context clean for each generation step.

### Phase 4: Composition and Integration

Once building blocks are verified, we compose them into full solutions. The microservice architecture means integration is primarily wiring well-tested components together. Because each component has a defined interface and verified behavior, composition is predictable and debuggable.

The productivity payoff is substantial. Solutions that would have taken months can come together in weeks—not because we cut corners, but because AI eliminated the mechanical burden of translating well-understood requirements into working code.

## Why This Works

The reason this approach succeeds where vibe coding often fails is simple alignment: We structure work to match what generative AI is actually good at.

Give it a clear specification, focused scope, and bounded problem space, and generated code quality is remarkably high. Give it a vague prompt and a sprawling codebase, and you get what critics rightly complain about: brittle, inconsistent code that appears to work until it catastrophically doesn't.

There is emerging empirical support for this principle. Borg, Bjarnason, and Hedin<sup>1</sup> observed that a vibe-coded prototype built with the Lovable platform—roughly 6,000 lines of TypeScript produced entirely through conversational prompting—scored 9.6 out of 10 on CodeScene's CodeHealth maintainability metric. The authors attribute this partly to Lovable's prompt engineering, which favors highly modular React components, and partly to the bounded

scope of the project itself. This reinforces the central point: When the problem space is constrained and the components are modular, AI-generated code quality can be remarkably high. The issue has never been the AI's coding ability; it's the size and clarity of the problem you give it.

This isn't novel. Decompose complex problems into manageable pieces, define clear interfaces, implement incrementally, verify continuously—these principles have governed effective software development for decades. The tool has changed. The engineering principles haven't.

### Humans in the Loop

There's a temptation, even among disciplined teams, to let the process become too automated. This needs to be addressed directly: effective AI-assisted development requires active human oversight at every phase.

During systems engineering, AI can propose architectures that look elegant but contain subtle flaws. A senior engineer isn't rubber-stamping; they're applying experiential judgment that no model possesses. When the AI gets it wrong, the human guides it: refining prompts, adding constraints, iterating until output reflects genuine engineering judgment.

AI-generated requirements can be impressively comprehensive and simultaneously miss the point—enumerating functional requirements while overlooking the one nonfunctional constraint that should have shaped the entire specification. Human review catches gaps that emerge from the model's lack of situational awareness.

The per-PR code generation phase is where human oversight pays its most visible dividends. Each increment needs review not just for functional correctness but for readability, maintainability, and robustness. When

code falls short, the engineer guides the AI toward better solutions. This iterative dialogue—not first-pass generation—is where production-grade code emerges.

As Simon Willison put it: "If an LLM wrote every line of your code, but you've reviewed, tested, and understood it all, that's not vibe coding in my book—that's using an LLM as a typing assistant."<sup>b</sup> The pattern across all phases is generation, validation, and guided refinement. Teams that skip this loop accumulate the technical debt the industry is rightly worried about.

### The Talent Pipeline Crisis

While the industry debates AI-generated code quality, quieter and arguably more consequential decisions are being made in hiring departments globally. Companies are simultaneously slashing junior engineering headcount and pushing experienced engineers toward the exits. Both moves are driven by the same flawed logic.

The junior hiring freeze is the more visible trend. According to SignalFire, entry-level hiring at major tech firms has dropped more than 50% over the past three years.<sup>c</sup> A Stanford Digital Economy Study found that employment for software developers aged 22–25 declined nearly 20% from its late 2022 peak.<sup>d</sup> Across the EU, junior tech positions fell 35% during 2024.<sup>c</sup> The reasoning seems straightforward: If AI can generate code, why hire entry-level engineers?

<sup>b</sup>S. Willison, quoted in "Vibe coding," Wikipedia. [Online]. Available: [https://en.wikipedia.org/wiki/Vibe\\_coding](https://en.wikipedia.org/wiki/Vibe_coding)

<sup>c</sup>AI is wiping out entry-level tech jobs, leaving graduates stranded," Rest of World, Dec. 19, 2025. [Online]. Available: <https://restofworld.org/2025/engineering-graduates-ai-job-losses/>

<sup>d</sup>AI vs Gen Z: How AI has changed the career pathway for junior developers," Stack Overflow Blog, Dec. 26, 2025. [Online]. Available: <https://stackoverflow.blog/2025/12/26/ai-vs-gen-z/>

But this fundamentally misunderstands what junior roles accomplish. Junior engineers aren't just cheap labor—they are the pipeline through which the industry develops its future senior engineers, architects, and technical leaders. The very people this article argues are essential at every step of AI-assisted development.

Less discussed but equally damaging is the pressure on experienced engineers. Across the industry, seasoned developers are being managed out, driven by assumptions that veterans can't adapt to AI workflows—or simply that they cost too much. This is particularly acute in some regions where negotiated workforce reductions disproportionately target older workers. Yet the engineers with the deepest architectural knowledge and most refined judgment are precisely the people best positioned to serve as the human-in-the-loop that effective AI-assisted development requires.

There's also an uncomfortable economic dynamic accelerating these decisions. Companies have made massive capital investments in AI infrastructure. Those investments create enormous pressure to demonstrate return on investment—and the fastest line item to cut is headcount. Organizations reduce engineering staff not because AI has absorbed those roles, but because the financial commitment to AI demands visible cost savings to justify the spend.

This puts the cart before the horse. The benefits of AI-assisted development are real, but they depend on having skilled engineers to execute disciplined workflows. Cutting the workforce to fund AI investment undermines the conditions needed for that investment to pay off.

## Developing Engineers in an AI World

For organizations still investing in talent development, there's a real pedagogical question. If AI handles most code generation, how do new engineers develop deep technical skills?

There is a meaningful difference between someone who has spent years writing code—wrestling with memory management, debugging concurrency issues, tracing data through complex call stacks—and someone who has primarily reviewed AI-generated output. The former develops an internalized model of how software works. The latter risks a superficial understanding that collapses when confronted with novel failure modes.

The answer isn't to reject AI tools. It's to be intentional. Structure early-career work so juniors still write code by hand for critical learning exercises—because writing, failing, debugging, and rewriting is how engineering intuition is forged. Require them to review and critique AI-generated code. Pair them with seniors during architectural phases.

Think of it as pilot training. Modern aviation uses extensive automation, but no airline certifies a pilot who has never hand-flown an aircraft. Software engineering needs the same philosophy: Let AI handle routine generation, but ensure humans have earned expertise through hands-on practice.

The productive response isn't eliminating roles—it's redesigning them. Juniors should spend less time on mechanical code production and more on systems thinking, architecture evaluation, and critical review of AI output. Seniors should



## ABOUT THE AUTHOR



**KURT ANTONY RICHARDSON** is head of Research & Development at Continuum Resources LLC, Phoenix, AZ 85340 USA. Contact him at [kurtr@continuumresourcesllc.com](mailto:kurtr@continuumresourcesllc.com).

be valued explicitly for the judgment and oversight capabilities that make AI-assisted workflows reliable.

### Getting It Right

This brings us back to where we started. The vibe coding critique is valid: Undisciplined AI-assisted development produces technical debt. But the response shouldn't be to abandon AI—it should be to apply the engineering discipline that has always been required for quality software.

Broy and Selić recently argued in this publication that software engineering is exceptionally susceptible to hype, and that the generative AI wave risks drawing attention away from foundational engineering principles.<sup>2</sup> I would argue something related but distinct: Generative AI, properly applied, reinforces those principles rather than replacing them. The teams succeeding with AI are the ones doing more systems engineering, more decomposition—not less. Meanwhile, Borg, Bjarnason, and Hedin<sup>1</sup> have shown that vibe coding, properly scoped as a prototyping practice, can deliver genuine value—lowering the threshold for high-fidelity prototypes that support communication, validation, and exploration. The key distinction is

purpose: prototyping to learn versus building to ship. The engineering discipline this article advocates isn't about rejecting rapid experimentation; it's about knowing when experimentation ends and production engineering begins.

**T**he companies that get this right will have a decisive advantage. The companies that treat AI as justification to hollow out their engineering organizations will find themselves staffed by people who can prompt an LLM but can't diagnose why the system it built is failing in production. And they will discover—too late—that they automated away the one capability they couldn't afford to lose.

The AI isn't the problem. The engineering is. It always has been. 🍷

### References

1. M. Borg, E. Bjarnason, and F. Hedin, "Vibe coding and the new prototyping playbook," *IEEE Softw.*, vol. 42, no. 6, pp. 12–16, Nov./Dec. 2025, doi: [10.1109/MS.2025.3597032](https://doi.org/10.1109/MS.2025.3597032).
2. M. Broy and B. Selić, "The effects of hype in the software domain: Causes, consequences, and mitigations," *IEEE Softw.*, vol. 42, no. 2, pp. 98–102, Mar./Apr. 2025, doi: [10.1109/MS.2024.3511732](https://doi.org/10.1109/MS.2024.3511732).